

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

«Встроенные системы»

Лабораторная работа №2

Целочисленный калькулятор на стенде SDK-1.1M

Вариант 1

Выполнил:

Козаченко Данил Александрович

Марьин Григорий Алексеевич

Группа Р3312

Проверил:

Гончаров Алексей Андреевич

Санкт-Петербург 2026г.

Содержание

1	Цель работы	2
2	Задание	2
3	Внесённые изменения в main.c	2
3.1	Запуск и опрос калькулятора	2
3.2	Обработчик ошибки	3
4	Диаграмма последовательности	3
5	Арифметическая логика калькулятора	5
5.1	Хранение состояния	5
5.2	Очистка состояния и текущее значение	5
5.3	Ввод чисел и выполнение операций	5
5.4	Контроль переполнения	6
6	Обработка ввода	6
6.1	Новая функция сканирования клавиш	6
6.2	Подавлениедребезга и длительность нажатия	7
6.3	Передача события калькулятору	7
7	Экран калькулятора	7
7.1	Формирование строк	7
7.2	Пример использования	8
8	Вывод	8
A	Основные фрагменты программы	9
A.1	Ввод цифр и вычисление результата	9
A.2	Опрос клавиатуры	10
A.3	Подавлениедребезга и обработка удержания	11

1 Цель работы

Разработать в `main.c` исходного проекта `SDK_Keyboard` логику целочисленного калькулятора: ввод операндов, выбор операции, вычисление результата и отображение состояния.

2 Задание

Реализовать вариант 1: калькулятор с операциями сложения, вычитания и умножения целых чисел. Ввод выполняется с клавиатуры стенда, данные и результат отображаются на дисплее.

3 Внесённые изменения в `main.c`

Исходный файл `SDK_Keyboard/Src/main.c` содержал тест клавиатуры. В выполненной работе он заменён логикой калькулятора. В пользовательской части файла внесены следующие изменения:

1. Подключены заголовки `stdint.h`, `limits.h`, `stdio.h` и `string.h` для целочисленных типов, границ диапазона, форматирования строк и очистки состояния.
2. Вместо объявлений `KB_Test()`, `OLED_KB()` и `oled_Reset()` добавлены объявления `Calculator_Init()` и `Calculator_Poll()`.
3. В `main()` добавлен вызов `Calculator_Init()`.
4. В главном цикле вызов `KB_Test()` заменён на `Calculator_Poll()`, задержка уменьшена с 500 до 5 мс.
5. Функции теста заменены структурой `Calculator`, переменной состояния `calc` и 12 функциями `Calculator_*`.
6. В `Error_Handler()` добавлены отключение прерываний и остановка выполнения в бесконечном цикле.

3.1 Запуск и опрос калькулятора

Добавленная функция `Calculator_Init()` очищает состояние, вызывает готовую функцию `Set_Keyboard()` и выводит начальный экран. Если настройка клавиатуры завершается ошибкой, она отображает `KEYBOARD I2C ERROR` и вызывает `Error_Handler()`.

Листинг 1: Добавленный вызов при запуске

```
1 Calculator_Init();
```

В теле главного цикла выполняется опрос калькулятора. Уменьшенная задержка позволяет чаще считывать изменения состояния клавиш.

Листинг 2: Новое содержимое тела главного цикла

```
1 Calculator_Poll();  
2 HAL_Delay(5);
```

3.2 Обработчик ошибки

Ранее пользовательская часть `Error_Handler()` была пустой. Теперь при её вызове прерывания отключаются, а выполнение останавливается:

Листинг 3: Добавленный код в `Error_Handler()`

```
1 __disable_irq();  
2 while (1) { }
```

4 Диаграмма последовательности

На рисунке 1 показано взаимодействие пользователя, клавиатуры, микроконтроллера и дисплея при работе калькулятора. Считывание состояний клавиш выполняет `Calculator_ReadKey()`, а подтверждение нажатия и определение его длительности — `Calculator_Poll()`. На диаграмме показаны события, обрабатываемые добавленными функциями калькулятора.

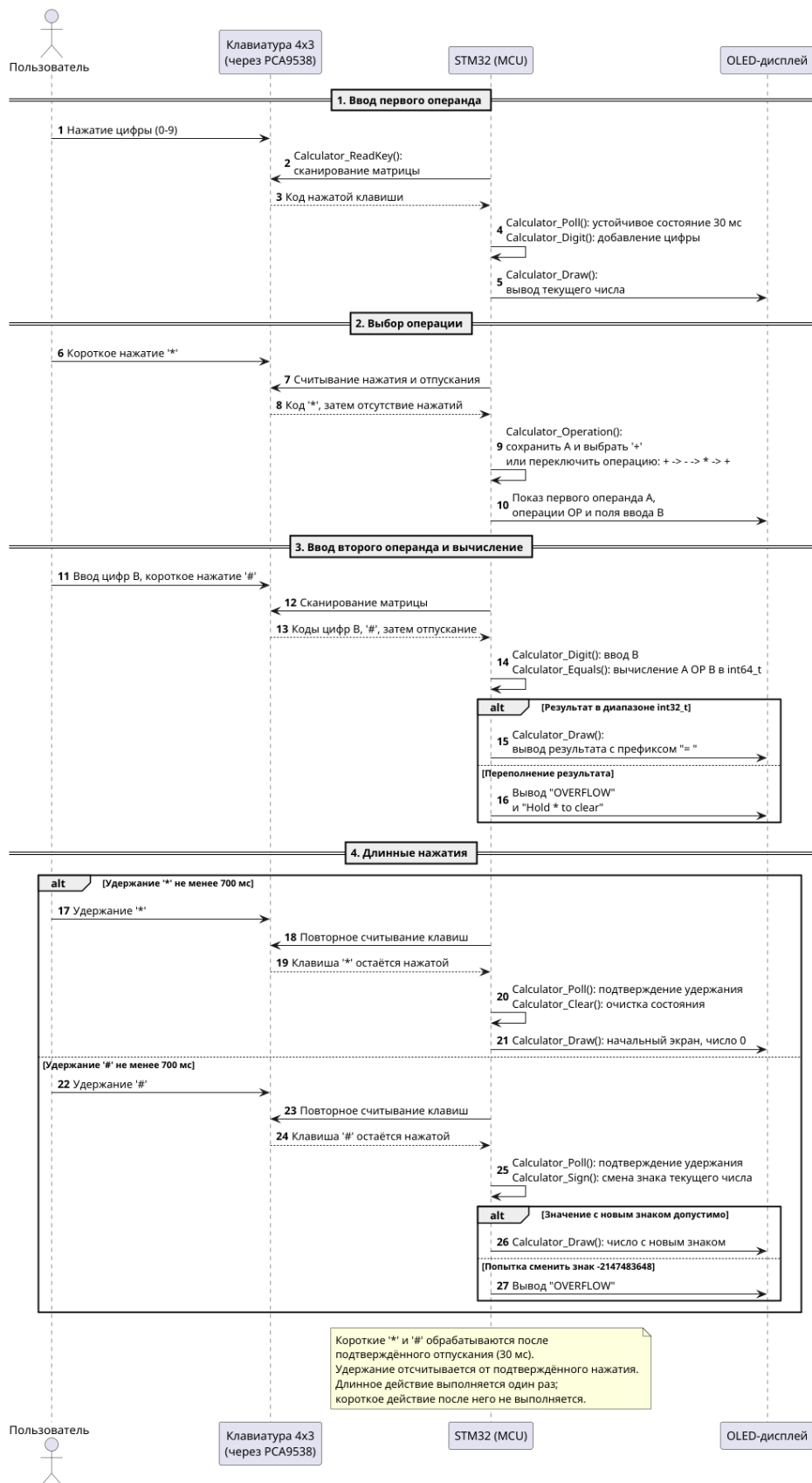


Рис. 1: UML-диаграмма последовательности работы калькулятора

Микроконтроллер постоянно сканирует клавиатуру; на диаграмме повторные опросы между пользовательскими действиями опущены. Короткие нажатия * и # обрабатываются после подтверждённого отпускания. Длительность удержания отсчитывается от

подтверждённого нажатия. Очистка выполняется через `Calculator_Clear()`, смена знака — через `Calculator_Sign()`, а строки дисплея формируются `Calculator_Draw()`.

5 Арифметическая логика калькулятора

5.1 Хранение состояния

Добавлена структура `Calculator` и статическая переменная `calc`, в которой хранится текущее состояние.

Таблица 1: Поля состояния калькулятора

Поле	Тип	Смысл
<code>left</code>	<code>int32_t</code>	Первый операнд или полученный результат
<code>magnitude</code>	<code>int64_t</code>	Модуль вводимого числа
<code>negative</code>	<code>uint8_t</code>	Признак отрицательного значения
<code>entered</code>	<code>uint8_t</code>	Введена хотя бы одна цифра
<code>pending</code>	<code>uint8_t</code>	Выбрана операция, вводится второй операнд
<code>result</code>	<code>uint8_t</code>	Отображается результат вычисления
<code>error</code>	<code>uint8_t</code>	Обнаружено переполнение
<code>operation</code>	<code>char</code>	Одна из операций <code>+</code> , <code>-</code> , <code>*</code>

5.2 Очистка состояния и текущее значение

Функция `Calculator_Clear()` обнуляет поля `calc` через `memset()` и устанавливает операцию сложения. Функция `Calculator_Value()` возвращает модуль со знаком, заданным полем `negative`.

5.3 Ввод чисел и выполнение операций

Функция `Calculator_Digit()` добавляет цифру d к модулю числа по формуле

$$m_{\text{new}} = 10m + d.$$

Для положительного значения разрешён модуль не более 2147483647, для отрицательного — не более 2147483648. Знак хранится отдельно, а текущее значение возвращается функцией `Calculator_Value()`. Для ввода -2147483648 сначала нужно установить отрицательный знак, затем вводить цифры модуля.

Функция `Calculator_Operation()` при первом коротком нажатии `*` после ввода числа сохраняет его в `left`, очищает поле ввода и выбирает сложение. Последующие короткие нажатия `*`, пока `pending=1`, переключают операции по циклу

$$+ \longrightarrow - \longrightarrow * \longrightarrow +.$$

Операцию можно изменить и после начала ввода второго операнда. Короткое нажатие `#` вызывает `Calculator_Equals()`. Результат вычисляется, только если операция выбрана и второй операнд введён. Неполное выражение игнорируется.

Таблица 2: Назначение клавиш

Клавиша	Действие	Результат
0–9	Нажатие	Добавление одной цифры
*	Короткое нажатие	Выбор или переключение операции
*	Удержание 700 мс	Полная очистка калькулятора
#	Короткое нажатие	Вычисление результата
#	Удержание 700 мс	Смена знака текущего числа

Функция `Calculator_Sign()` меняет знак текущего числа. При отображении результата она также обновляет `left`, чтобы следующий расчёт использовал значение с новым знаком.

После вычисления результат можно использовать как первый операнд: короткое нажатие `*` начинает следующий расчёт. Ввод цифры после результата или переполнения автоматически начинает новое число. Длинное нажатие `*` сбрасывает все поля; операцией по умолчанию снова становится сложение.

5.4 Контроль переполнения

Вводимые значения и итоговый результат ограничены диапазоном

$$-2^{31} \leq x \leq 2^{31} - 1.$$

При вычислении первый операнд преобразуется в `int64_t` до сложения, вычитания или умножения. Это позволяет проверить полученное значение перед приведением к `int32_t`. Произведение любых двух 32-разрядных операндов помещается в знаковый 64-разрядный промежуточный результат.

Если новый модуль, арифметический результат или смена знака -2147483648 выходят за допустимый диапазон, устанавливается `error=1` и отображается `OVERFLOW`. До очистки или ввода новой цифры выбор операции и вычисление игнорируются; смена знака также не выполняется.

6 Обработка ввода

6.1 Новая функция сканирования клавиш

Добавлена `Calculator_ReadKey()`, которая за один вызов проверяет все четыре строки и возвращает символ выбранной клавиши. Внутри неё задано соответствие позиций матрицы символам: 1, 2, 3 / 4, 5, 6 / 7, 8, 9 / *, 0, #.

Функция последовательно выбирает строки с помощью готовых констант `ROW1–ROW4`. Для выбранной строки используется режим выхода с нулевым уровнем, остальные строки остаются входами. После переключения предусмотрена задержка 1 мс, затем считываются три столбца и подсчитывается число обнаруженных нажатий. При успешном завершении все выводы переводятся обратно в режим входа.

В новой функции проверяется результат каждого вызова готовых операций `PCA9538_Write_Register()` и `PCA9538_Read_Inputs()`. Возвращаемое значение имеет следующий смысл:

- 0: клавиши не нажаты;
- символ цифры, * или #: нажата одна клавиша;
- 0xFF: обнаружены несколько клавиш или ошибка обмена.

6.2 Подавлениедребезга и длительность нажатия

Функция `Calculator_Poll()` запоминает считанное значение `candidate` и момент его изменения `changed_at`. Новое устойчивое состояние `stable` принимается после 30 мс без изменения считанного значения.

При подтверждённом нажатии цифры формируется одно событие ввода. Удержание не повторяет цифру. Для клавиш * и # короткое действие выполняется после подтверждённого отпускания, а длинное — после 700 мс удержания, отсчитываемых от подтверждённого нажатия. Флаг `long_done` позволяет выполнить длинное действие один раз и исключает короткое действие при последующем отпускании.

Если сканирование вернуло 0xFF или клавиша сменилась без полного отпускания, устанавливается `blocked`. Обработка возобновляется только после отпускания всех клавиш. Для отсчёта интервалов новая функция использует `HAL_GetTick()`.

6.3 Передача события калькулятору

Добавленная функция `Calculator_Key()` получает код клавиши и признак удержания `held`. Она выбирает арифметическое действие и после его выполнения обновляет экран.

Листинг 4: Распределение действий по клавишам

```
1 static void Calculator_Key(uint8_t key, uint8_t held)
2 {
3     if (key >= '0' && key <= '9') Calculator_Digit(key - '0');
4     else if (key == '*') {
5         if (held) Calculator_Clear();
6         else Calculator_Operation();
7     } else if (key == '#') {
8         if (held) Calculator_Sign();
9         else Calculator_Equals();
10    }
11    Calculator_Draw();
12 }
```

7 Экран калькулятора

7.1 Формирование строк

Добавлена `Calculator_Line()`, которая размещает строку в позиции $x = 0$ с заданной координатой y , вызывая готовые `oled_SetCursor()` и `oled_WriteString()`.

Добавленная функция `Calculator_Draw()` очищает буфер дисплея, выводит состояние и вызывает `oled_UpdateScreen()`. Применяется шрифт `Font_7x10`. В строке шириной

128 точек помещается 18 символов; локальный буфер `line[19]` содержит также завершающий нулевой байт.

В верхней строке ($y = 0$) выводится `INTEGER CALC`. При вводе первого операнда строка $y = 12$ содержит текущее число. При выборе операции строки $y = 12, 24, 36$ показывают первый операнд с префиксом `A:`, операцию `OP:` и второй операнд `B:`. После расчёта результат имеет префикс `=`. В нижней строке ($y = 52$) постоянно отображается подсказка `Hold: * C # +/-`.

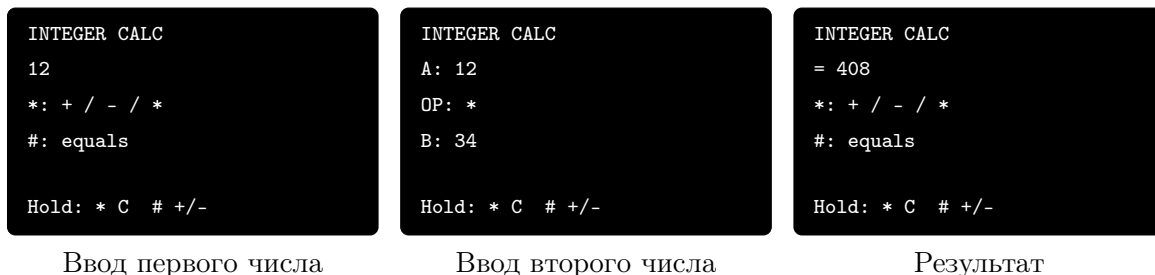


Рис. 2: Схематическое воспроизведение экранов по функции `Calculator_Draw()`, пример $12 \cdot 34 = 408$

Функция `Calculator_Draw()` вызывается при инициализации и после обработанного действия пользователя. При переполнении выводятся строки `OVERFLOW` и `Hold * to clear`.

7.2 Пример использования

Чтобы вычислить $12 \cdot 34$, следует ввести 1, 2, трижды коротко нажать `*` (выбор сложения, затем вычитания и умножения), ввести 3, 4 и коротко нажать `#`. На дисплее появится $= 408$.

Для $-12 + 34$ перед вводом первого числа следует удерживать `#` до смены знака, ввести 1, 2, коротко нажать `*`, ввести 3, 4 и коротко нажать `#`. Результат равен 22. Для нового расчёта можно удерживать `*` либо начать ввод цифр после отображения результата.

8 Вывод

В пользовательской части `main.c` реализована логика целочисленного калькулятора: хранение состояния, ввод операндов, выбор сложения, вычитания и умножения, вычисление результата и контроль переполнения.

Добавлены сканирование всей клавиатуры за один проход, подавление дребезга, различение короткого нажатия и удержания, блокировка неоднозначного ввода, очистка и смена знака. Разработаны экраны ввода и результата. Калькулятор подключён к запуску программы и главному циклу; обработчик ошибки дополнен остановкой выполнения.

А Основные фрагменты программы

Ниже приведены фрагменты предоставленного `main.c`. Полный код можно посмотреть на [GitHub](#)

А.1 Ввод цифр и вычисление результата

Листинг 5: Добавление цифры и проверка диапазона

```
1 static void Calculator_Digit(uint8_t digit)
2 {
3     if (calc.error || calc.result) Calculator_Clear();
4     int64_t next = calc.magnitude * 10 + digit;
5     int64_t limit = calc.negative ? (int64_t)INT32_MAX + 1 : INT32_MAX;
6     if (next > limit) {
7         calc.error = 1;
8         return;
9     }
10    calc.magnitude = next;
11    calc.entered = 1;
12 }
```

Листинг 6: Арифметические операции и проверка результата

```
1 static void Calculator_Equals(void)
2 {
3     if (calc.error || !calc.pending || !calc.entered) return;
4     int64_t right = Calculator_Value();
5     int64_t answer;
6     switch (calc.operation) {
7         case '+': answer = (int64_t)calc.left + right; break;
8         case '-': answer = (int64_t)calc.left - right; break;
9         default: answer = (int64_t)calc.left * right; break;
10    }
11    if (answer < INT32_MIN || answer > INT32_MAX) {
12        calc.error = 1;
13        return;
14    }
15    calc.left = (int32_t)answer;
16    calc.negative = answer < 0;
17    calc.magnitude = answer < 0 ? -answer : answer;
18    calc.pending = 0;
19    calc.result = 1;
20 }
```

A.2 Опрос клавиатуры

Листинг 7: Сканирование матрицы и контроль статусов обмена

```
1 static uint8_t Calculator_ReadKey(void)
2 {
3     static const uint8_t rows[4] = {ROW1, ROW2, ROW3, ROW4};
4     static const uint8_t keys[4][3] = {{'1','2','3'}, {'4','5','6'},
5                                         {'7','8','9'}, {'*','0','#'}};
6     uint8_t found = 0, count = 0, input = 0x70;
7     for (uint8_t row = 0; row < 4; ++row) {
8         uint8_t config = rows[row];
9         if (PCA9538_Write_Register(0xE2, CONFIG, &config) != HAL_OK)
10             return 0xFF;
11         HAL_Delay(1);
12         if (PCA9538_Read_Inputs(0xE2, &input) != HAL_OK) return 0xFF;
13         for (uint8_t column = 0; column < 3; ++column) {
14             if (!(input & (0x10u << column))) {
15                 found = keys[row][column];
16                 ++count;
17             }
18         }
19     }
20     uint8_t idle = 0xFF;
21     if (PCA9538_Write_Register(0xE2, CONFIG, &idle) != HAL_OK) return 0xFF;
22     return count > 1 ? 0xFF : found;
23 }
```

A.3 Подавление дребезга и обработка удержания

Листинг 8: Формирование событий короткого и длинного нажатия

```
1 static void Calculator_Poll(void)
2 {
3     static uint8_t candidate, stable, pressed, long_done, blocked;
4     static uint32_t changed_at, pressed_at;
5     uint8_t raw = Calculator_ReadKey();
6     uint32_t now = HAL_GetTick();
7     if (raw == 0xFF) {
8         blocked = 1;
9         pressed = 0;
10    }
11    if (raw != candidate) {
12        candidate = raw;
13        changed_at = now;
14    }
15    if (now - changed_at >= 30 && stable != candidate) {
16        stable = candidate;
17        if (!stable) {
18            if (pressed && !long_done && !blocked &&
19                (pressed == '*' || pressed == '#'))
20                Calculator_Key(pressed, 0);
21            pressed = 0;
22            blocked = 0;
23        } else if (!blocked) {
24            if (pressed) {
25                blocked = 1;
26                pressed = 0;
27            } else {
28                pressed = stable;
29                pressed_at = now;
30                long_done = 0;
31                if (pressed >= '0' && pressed <= '9') Calculator_Key(pressed, 0);
32            }
33        }
34    }
35    if (pressed && !blocked && !long_done && raw == pressed &&
36        (pressed == '*' || pressed == '#') && now - pressed_at >= 700) {
37        long_done = 1;
38        Calculator_Key(pressed, 1);
39    }
40 }
```

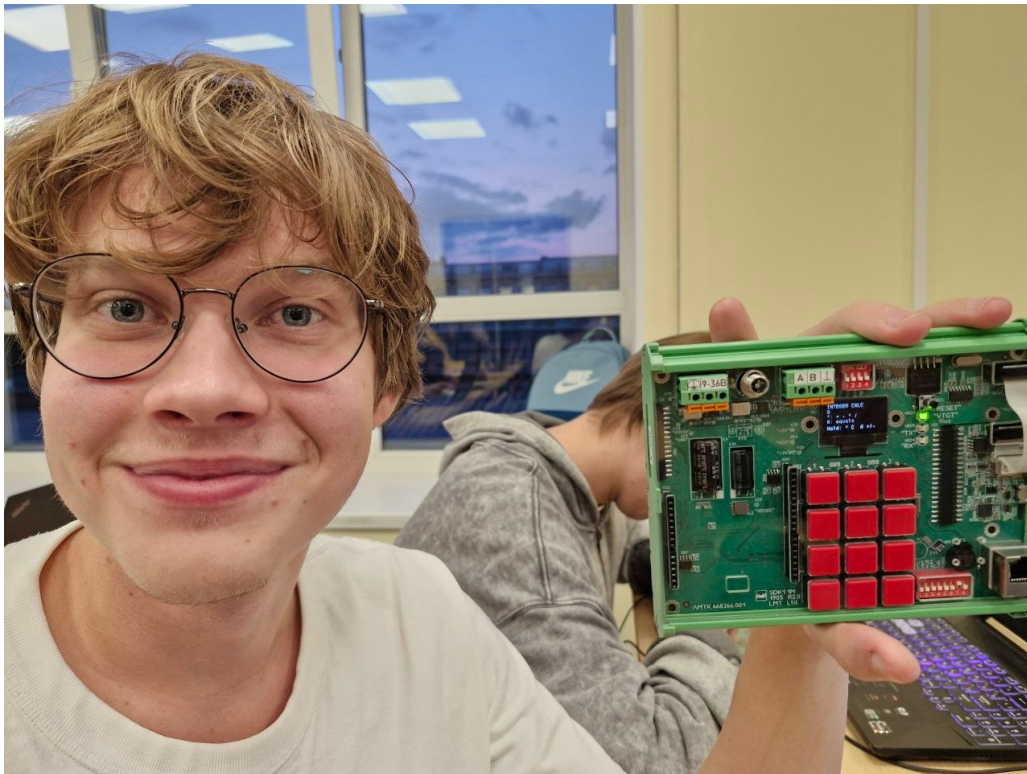


Рис. 3: Запущенная программа на плате

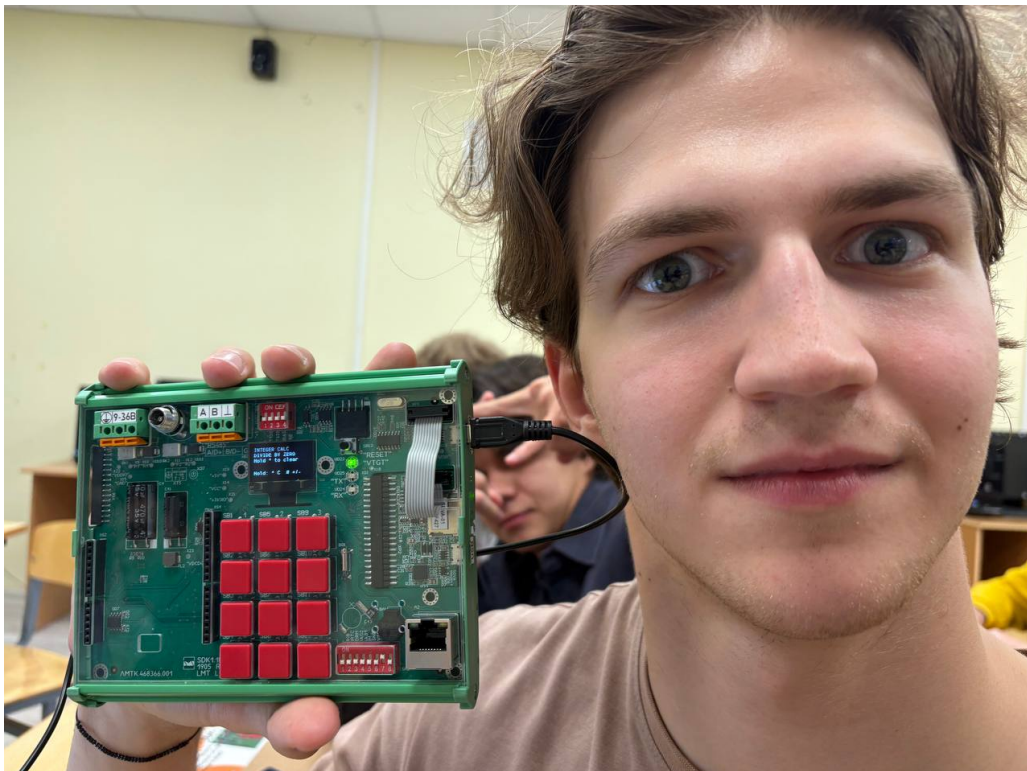


Рис. 4: Деление на 0 на плате